Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования «Сибирский государственный университет телекоммуникаций и информатики»

(СибГУТИ)

Кафедра прикладной математики и кибернетики

КУРСОВАЯ РАБОТА

по дисциплине «Операционные системы»

Тема: «Разработка сетевого приложения, реализующего сетевой чат на основе сокетов»

Выполнили:
студенты гр. ИП-111 Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Работу принял
Профессор кафедры прикладной
математики и кибернетики _____/Малков Е.А./

«18» января 2023г. Оценка_____.

г. Новосибирск 2023

Оглавление

Введение: 3

1. Постановка задачи 4

1.1. Анализ предметной области 4

1.2. Цель создания 4

1.3. Перечень решаемых задач 4

2. Программное проектирование 5

2.1. Используемое программное обеспечение 5

2.2. Определение функциональной части 5

3. Программная реализация 6

3.1. Описание программы 6

Клиент 6

Сервер 7

4. Пример работы программы 10

5. Личный вклад каждого участника бригады 11

6. Заключение 12

7. Список использованных источников 13

8. Листинг 14

Клиент 14

Сервер 28

Введение:

Работая в сети Ethernet, мы очень часто встречаемся с разного рода многопользовательскими программами. Ими могут быть почтовые клиенты, чаты, форумы, FTP клиенты и т.п. Все эти приложения используют для своей работы различные протоколы: FTP, POP, SMTP, HTTP, и т.д. Но базовым для них является единый протокол - TCP/IP. Типичное же приложение TCP/IP построено на клиент-серверной архитектуре. Примером приложения, построенного на данной архитектуре, является многопользовательский чат. При работе с протоколом TCP/IP используют интерфейс сокетов. Но, прежде чем перейти к разработке чата, разберем, что же такое сокеты.

Сокеты – это интерфейс прикладного программирования для сетевых приложений TCP/IP. В переводе с английского "sockets" – гнезда, т.е. сетевые приложения используют сокеты, как виртуальные разъемы для обмена данными между собой. Сокеты бывают трех видов: клиентские, слушающие и серверные. Клиентские сокеты устанавливают связь с сервером и обмениваются с ним данными. Серверный сокет обменивается данными с клиентом по уже установленному (слушающим сокетом) соединению. Для того чтобы клиент мог установить соединение с сервером, ему необходимо указать его адрес (IP) и номер порта, через который будет происходить обмен данными.

В данной курсовой работе предполагается реализовать программное обеспечение с графическим интерфейсом для обмена сообщениями посредством сети. Основное предназначение реализуемого программного продукта - передача символьной информации. Язык, выбранный для реализации курсового проекта – C++ - язык программирования, подходящий для решения множества различных задач.

Данный программный продукт может использоваться в качестве корпоративного приложения, например для обмена сообщениями между офисными работниками, в многопользовательских играх, образовательных платформах и т.д.

1. Постановка задачи

В качестве курсовой работы мы выбрали тему - разработка сетевого приложения, реализующего сетевой чат на основе сокетов.

1.1. Анализ предметной области

Предметная область проекта - передача информации по сети. Чат, позволяющий организовать общение между неограниченным количеством пользователей.

Требуется разработать приложение, которое включает в себя разработку следующих функциональных частей: разработку клиент-серверного приложения.

1.2. Цель создания

Главная цель курсовой работы по теме «Многопользовательский сетевой чат» – создание многопользовательского сетевого чата с возможностью обмена текстовыми сообщениями между пользователями, подключенными к одному серверу.

1.3. Перечень решаемых задач

Функционал данного сетевого чата предусматривает выполнение следующих задач:

- Подключение двух и более пользователей к серверу для возможности общения друг с другом;
- Обмен сообщений между клиентами;

2. Программное проектирование

2.1. Используемое программное обеспечение

При написании данного курсового проекта планируется использовать следующее стороннее программное обеспечение и инструментарий:

Qt — кроссплатформенный IDE для разработки программного обеспечения на языке программирования C++.

QtWidgets — компонент библиотеки Qt, содержит классы для классических приложений на основе виджетов.

QTcpSocket — вспомогательный подкласс QAbstractSocket, который позволяет устанавливать TCP-соединение и передавать потоки данных.

QTcpServer — класс, который предназначен для управления входящими TCP-соединениями.

Разработка будет осуществляться в операционной системе Ubuntu 22.04. Серверное приложение будет реализовано с интерфейсом для более комфортного и доступного использования.

2.2. Определение функциональной части

Прежде, чем начинать разработку требуемый функционал нашего программного обеспечения.

Данный проект можно разделить на две части:

1. клиентское приложение
2. серверная часть

Пользователь через клиентское приложение взаимодействует непосредственно с приложением-сервером и должен иметь возможность выполнить следующие действия:

- Присоединиться к серверу;
- Отослать сообщение пользователю/группе пользователей;
- Принимать сообщения от других пользователей.

3. Программная реализация

Конечный программный продукт состоит из двух частей: сервер и клиент, запуск производится с помощью одноименных Executable файлов: client, server.

3.1. Описание программы

Клиент

dialog.cpp

Dialog::Dialog(QWidget *parent) :QDialog(parent),ui(new Ui::Dialog):

конструктор класса Dialog. В нем происходит вывод окна с интерфейсом для подключения к чату, подключение сигналов QTcpSocket::readyRead connected, disconnected, error(QAbstractSocket::SocketError) к соответствующим слотам обработки.

void Dialog::onSokDisplayError(QAbstractSocket::SocketError socketError):

обработка ошибок, связанных с подключением к сокету.

void Dialog::onSokReadyRead():

обработчик вывода сообщений в основное окно чата на основе полученной команды и дополнительных данных.

void Dialog::onSokConnected():

обработка подключения к сокету и попытка авторизовать пользователя

void Dialog::onSokDisconnected():

обработка отключения пользователя от сокета.

void Dialog::on_pbConnect_clicked(),

void Dialog::on_pbDisconnect_clicked():

обработка нажатия кнопки подключения / отключения

void Dialog::on_cbToAll_clicked():

смена режима отправки сообщния между «всем» и «личное»

void Dialog::on_pbSend_clicked():

обработка нажатие кнопки отправки сообщения

void Dialog::addToLog(QString text, QColor color):

метод вывода всех сообщений на QListWidget в нужном формате.

Сервер

dialog.cpp

Dialog::Dialog(QWidget *parent) :QDialog(parent),ui(new Ui::Dialog):

конструктор класса Dialog. В нем происходит вывод окна с интерфейсом сервера, подключение сигналов messageFromGui, addLogToGui к соответствующим слотам обработки и запуск сервера на локальном хосте.

void Dialog::onAddUserToGui(QString name):

визуальное добавление подключенных пользователей в userlist сервера

void Dialog::onRemoveUserFromGui(QString name):

визуальное удаление подключенных пользователей из userlist сервера

void Dialog::onMessageToGui(QString message, QString from, const QStringList &users):

обработчик вывода сообщений всех пользователей в чат сервера

void Dialog::onAddLogToGui(QString string, QColor color):

обработчик вывода информационных выводов

void Dialog::on_pbSend_clicked():

обработчик нажатия кнопки отправки сообщения

void Dialog::on_cbToAll_clicked():

смена режима отправки сообщений

void Dialog::on_pbStartStop_toggled(bool checked):

обработка кнопки включения/ выключения сервера

void Dialog::addToLog(QString text, QColor color):

метод вывода всех сообщений на QListWidget в нужном формате.

myserver.cpp

bool MyServer::doStartServer(QHostAddress addr, qint16 port):

проверка запустился ли сервер

void MyServer::doSendToAllUserJoin(QString name):

отправляем клиентам список всех подключенных пользователей

void MyServer::doSendToAllUserLeft(QString name):

отправляем клиентам список всех оставшихся пользователей

void MyServer::doSendToAllMessage(QString message, QString fromUsername):

отправляем клиентам сообщение от пользователя

void MyServer::doSendMessageToUsers(QString message, const QStringList &users, QString fromUsername):

отправляем клиентам приватное сообщение от пользователя

void MyServer::doSendToAllServerMessage(QString message):

отправляем клиентам сообщение от сервера

void MyServer::doSendToServerMessageToUsers(QString message):

отправляем клиентам приватное сообщение от сервера

QStringList MyServer::getUsersOnline() const:

получение списка пользователей

bool MyServer::isNameValid(QString name) const:

проверка имени пользователя на подходящие знаки

bool MyServer::isNameUsed(QString name) const:

проверка имени пользователя на повторное существование в сети

void MyServer::incomingConnection(qintptr handle):

при входящем подключении создаём новый объект клиента и связываем с ним сигналы визуального обновления окон сервера, затем записываем нового пользователя в список пользователей

void MyServer::onRemoveUser(MyClient *client):

удаляет покинувшего пользователя из списка пользователя

void MyServer::onMessageFromGui(QString message, const QStringList &users):

в зависимости от выбранных пользователей выбираем как отправлять сообщение клиентам

myclient.cpp

MyClient::MyClient(int desc, MyServer *serv, QObject *parent) :QObject(parent):

аналог конструктора dialog.cpp из клиентской части.

void MyClient::onDisconnect():

обработка отсоединения пользователя

void MyClient::onReadyRead():

обработка запросов от клиента

void MyClient::doSendCommand(quint8 comm) const:

отправка клиенту кода ошибки в случае неудачной авторизации

void MyClient::doSendUsersOnline() const:

отправка клиентам списка пользователей в сети

4. Пример работы программы



Рис. 1 – Приложение сервера.

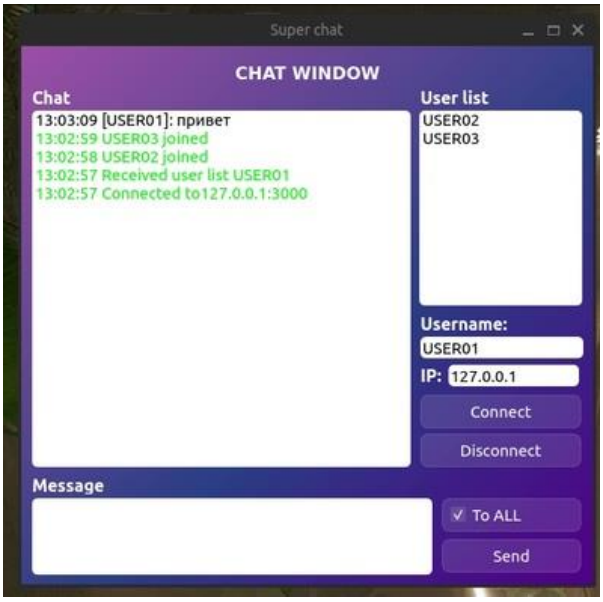


Рис. 2 – Приложение клиента (USER01).

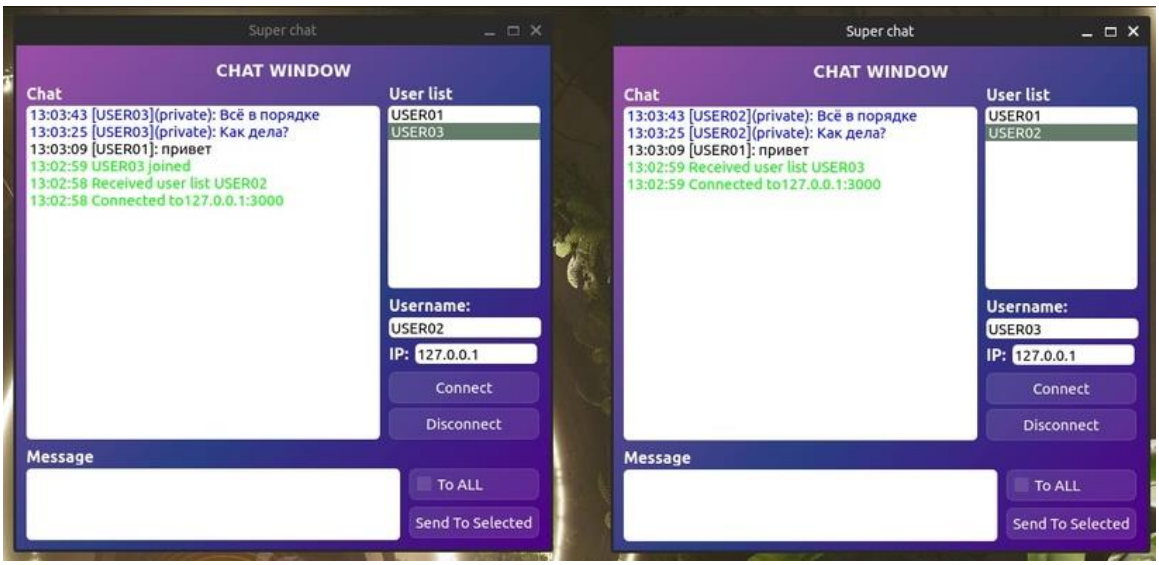


Рис. 3 – Приложение клиента (USER02, USER03).

Пояснение к изображениям:

1. Был запущен локальный сервер по IP адресу: 127.0.0.1:3000.
2. После запуска сервера к нему последовательно подключились 3 пользователя (USER01, USER02, USER03).
3. USER01 отправил сообщение с текстом “привет” всем участникам чата.
4. USER02 отправил сообщение “Как дела?” пользователю с ником USER03. Данное сообщение отобразилось в окне клиента USER02 и USER03. Окно пользователя USER01 осталось без изменений.
5. USER03 ответил USER02 сообщением “Все в порядке”. Данное сообщение так же отобразилось в окне клиента USER02 и USER03. Окно пользователя USER01 осталось без изменений.
6. Все подключения и сообщения логируются в окне администратора сервера. Администратор сервера также, как и обычный пользователь может отправлять сообщения другим пользователям.

5. Личный вклад каждого участника бригады

Гердележов Даниил:

Клиентская часть приложения (backend).

Создание UI интерфейса клиента.

Отчет по выполненной работе.

Махаева Алина:

Серверная часть приложения (backend).

Создание UI интерфейса сервера.

Клепче Георгий:

Серверная часть приложения (backend).

Подключение UI интерфейсов клиента и сервера к backend части проекта.

6. Заключение

На основе полученных знаний по курсу «Операционные системы» нам в результате удалось выполнить курсовую работу по теме «Многопользовательский сетевой чат на основе сокетов»;

Было разработано приложение-клиент на основе QTcpSocket, которое имело графический интерфейс, написанный с помощью QtWidget.

Разработанный нами программный продукт позволяет удалённо обмениваться информацией и данными в локальной сети и сети Ethernet.

7. Список использованных источников

1. Курс лекций Малкова Е.А. «Операционные системы».
2. MDN Web Docs *“WebSockets”* [электронный ресурс] // https://developer.mozilla.org/ru/docs/Web/API/WebSockets_API
3. IBM “Протоколы TCP/IP” [электронный ресурс] // https://www.ibm.com/docs/ru/aix/7.2?topic=protocol-tcpip-protocols
4. Олифер Виктор, Олифер Наталья *“Компьютерные сети. Принципы, технологии, протоколы: Юбилейное издание.”* — СПб.: Питер, 2020. — 1008 с.: ил. — (Серия «Учебник для вузов»). ISBN 978-5-4461-1426-9

8. Листинг

Клиент

dialog.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>Dialog</class>
 <widget class="QWidget" name="Dialog">
  <property name="geometry">
   <rect>
    <x>0</x>
    <y>0</y>
    <width>530</width>
    <height>500</height>
   </rect>
  </property>
  <property name="minimumSize">
   <size>
    <width>530</width>
    <height>500</height>
   </size>
  </property>
  <property name="maximumSize">
   <size>
    <width>530</width>
    <height>500</height>
   </size>
  </property>
  <property name="windowTitle">
   <string>Super chat</string>
  </property>
  <property name="styleSheet">
   <string notr="true">background: qlineargradient(spread:pad, x1:1, y1:1, x2:0,
y2:0, stop:0 rgba(81, 0, 135, 255), stop:0.427447 rgba(41, 61, 132, 235), stop:1
rgba(155, 79, 165, 255));
</string>
  </property>
  <widget class="QListWidget" name="lwLog">
   <property name="geometry">
    <rect>
     <x>10</x>
     <y>60</y>
     <width>351</width>
     <height>331</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">background:  rgb(255, 255, 255);
```

```
border-radius: 5px;</string>
   </property>
   <property name="resizeMode">
    <enum>QListView::Fixed</enum>
   </property>
  </widget>
  <widget class="QListWidget" name="lwUsers">
   <property name="geometry">
    <rect>
     <x>369</x>
     <y>60</y>
     <width>151</width>
     <height>181</height>
    </rect>
   </property>
   <property name="minimumSize">
    <size>
     <width>140</width>
     <height>160</height>
    </size>
   </property>
   <property name="styleSheet">
    <string notr="true">background:  rgb(255, 255, 255);
color: black;
border-radius: 5px;</string>
   </property>
   <property name="selectionMode">
    <enum>QAbstractItemView::MultiSelection</enum>
   </property>
   <property name="sortingEnabled">
    <bool>true</bool>
   </property>
  </widget>
  <widget class="QPlainTextEdit" name="pteMessage">
   <property name="geometry">
    <rect>
     <x>10</x>
     <y>420</y>
     <width>371</width>
     <height>71</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">background:  rgb(255, 255, 255);
color: black;
border-radius: 5px;</string>
   </property>
  </widget>
  <widget class="QLabel" name="msg_label">
   <property name="geometry">
    <rect>
```

```
      <x>10</x>
      <y>400</y>
      <width>81</width>
      <height>16</height>
     </rect>
    </property>
    <property name="styleSheet">
     <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
    </property>
    <property name="text">
     <string>Message</string>
    </property>
   </widget>
   <widget class="QLabel" name="chat_label">
    <property name="geometry">
     <rect>
      <x>10</x>
      <y>40</y>
      <width>81</width>
      <height>16</height>
     </rect>
    </property>
    <property name="styleSheet">
     <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
    </property>
    <property name="text">
     <string>Chat</string>
    </property>
   </widget>
   <widget class="QLabel" name="usr_label">
    <property name="geometry">
     <rect>
      <x>370</x>
      <y>40</y>
      <width>141</width>
      <height>16</height>
     </rect>
    </property>
    <property name="styleSheet">
     <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
```

```
border: none;</string>
   </property>
   <property name="text">
    <string>User list</string>
   </property>
  </widget>
  <widget class="QLabel" name="label_3">
   <property name="geometry">
    <rect>
     <x>10</x>
     <y>10</y>
     <width>511</width>
     <height>31</height>
    </rect>
   </property>
   <property name="font">
    <font>
     <family>Monotype Corsiva</family>
     <pointsize>12</pointsize>
     <bold>true</bold>
    </font>
   </property>
   <property name="layoutDirection">
    <enum>Qt::LeftToRight</enum>
   </property>
   <property name="styleSheet">
    <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
   </property>
   <property name="text">
    <string>CHAT WINDOW</string>
   </property>
   <property name="alignment">
    <set>Qt::AlignCenter</set>
   </property>
  </widget>
  <widget class="QCheckBox" name="cbToAll">
   <property name="enabled">
    <bool>false</bool>
   </property>
   <property name="geometry">
    <rect>
     <x>390</x>
     <y>420</y>
     <width>129</width>
     <height>31</height>
    </rect>
   </property>
```

```
   <property name="styleSheet">
    <string notr="true">QCheckBox{
    padding-left: 7px;
    color: rgb(255, 255, 255);
    background-color:rgba(255,255,255,30);
    border: 1px solid rgba(255,255,255,40);
    border-radius:7px;
}</string>
   </property>
   <property name="text">
    <string>To ALL</string>
   </property>
   <property name="checked">
    <bool>true</bool>
   </property>
  </widget>
  <widget class="QPushButton" name="pbSend">
   <property name="enabled">
    <bool>false</bool>
   </property>
   <property name="geometry">
    <rect>
     <x>390</x>
     <y>458</y>
     <width>129</width>
     <height>31</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">QPushButton{
    color: rgb(255, 255, 255);
    background-color:rgba(255,255,255,30);
    border: 1px solid rgba(255,255,255,40);
    border-radius:7px;
}

QPushButton:hover{
    background-color:rgba(255,255,255,30);
}

QPushButton:pressed{
    background-color:rgba(255,255,255,70);
}</string>
   </property>
   <property name="text">
    <string>Send</string>
   </property>
  </widget>
  <widget class="QLineEdit" name="leName">
   <property name="geometry">
    <rect>
```

```
      <x>370</x>
      <y>270</y>
      <width>151</width>
      <height>20</height>
     </rect>
    </property>
    <property name="styleSheet">
     <string notr="true">background:  rgb(255, 255, 255);
color: black;
border-radius: 5px;</string>
    </property>
    <property name="text">
     <string>MyName</string>
    </property>
   </widget>
   <widget class="QLineEdit" name="leHost">
    <property name="geometry">
     <rect>
      <x>396</x>
      <y>296</y>
      <width>121</width>
      <height>20</height>
     </rect>
    </property>
    <property name="styleSheet">
     <string notr="true">background:  rgb(255, 255, 255);
color: black;
border-radius: 5px;</string>
    </property>
    <property name="text">
     <string>127.0.0.1</string>
    </property>
   </widget>
   <widget class="QPushButton" name="pbConnect">
    <property name="geometry">
     <rect>
      <x>370</x>
      <y>324</y>
      <width>149</width>
      <height>31</height>
     </rect>
    </property>
    <property name="styleSheet">
     <string notr="true">QPushButton{
     color: rgb(255, 255, 255);
     background-color:rgba(255,255,255,30);
     border: 1px solid rgba(255,255,255,40);
     border-radius:7px;
}

QPushButton:hover{
```

```
    background-color:rgba(255,255,255,30);
}

QPushButton:pressed{
    background-color:rgba(255,255,255,70);
}</string>
   </property>
   <property name="text">
    <string>Connect</string>
   </property>
  </widget>
  <widget class="QPushButton" name="pbDisconnect">
   <property name="enabled">
    <bool>false</bool>
   </property>
   <property name="geometry">
    <rect>
     <x>370</x>
     <y>360</y>
     <width>149</width>
     <height>31</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">QPushButton{
    color: rgb(255, 255, 255);
    background-color:rgba(255,255,255,30);
    border: 1px solid rgba(255,255,255,40);
    border-radius:7px;
}

QPushButton:hover{
    background-color:rgba(255,255,255,30);
}

QPushButton:pressed{
    background-color:rgba(255,255,255,70);
}</string>
   </property>
   <property name="text">
    <string>Disconnect</string>
   </property>
  </widget>
  <widget class="QLabel" name="usr_label_2">
   <property name="geometry">
    <rect>
     <x>370</x>
     <y>250</y>
     <width>141</width>
     <height>16</height>
    </rect>
```

```
   </property>
   <property name="styleSheet">
    <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
   </property>
   <property name="text">
    <string>Username:</string>
   </property>
  </widget>
  <widget class="QLabel" name="usr_label_3">
   <property name="geometry">
    <rect>
     <x>370</x>
     <y>296</y>
     <width>31</width>
     <height>20</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
   </property>
   <property name="text">
    <string>IP:</string>
   </property>
  </widget>
 </widget>
 <layoutdefault spacing="6" margin="11"/>
 <resources/>
 <connections/>
</ui>
```

main.cpp

```
#include "dialog.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    Dialog w;
    w.show();

    return a.exec();
}
```

dialog.cpp

```
#include "dialog.h"
#include "ui_dialog.h"

#include <QtGui>
#include <QDebug>

#define PORT 3000

Dialog::Dialog(QWidget *parent) :QDialog(parent),ui(new Ui::Dialog)
{
    ui->setupUi(this);
    setWindowIcon(QIcon("favicon.PNG"));
    _name = "";
    //создаем сокет
    _sok = new QTcpSocket(this);
    //подключаем сигналы
    connect(_sok, SIGNAL(readyRead()), this, SLOT(onSokReadyRead()));
    connect(_sok, SIGNAL(connected()), this, SLOT(onSokConnected()));
    connect(_sok, SIGNAL(disconnected()), this, SLOT(onSokDisconnected()));
    connect(_sok, SIGNAL(error(QAbstractSocket::SocketError)),this,
SLOT(onSokDisplayError(QAbstractSocket::SocketError)));
}

Dialog::~Dialog()
{
    delete ui;
}

void Dialog::onSokDisplayError(QAbstractSocket::SocketError socketError)
{
    switch (socketError) {
    case QAbstractSocket::RemoteHostClosedError:
        break;
    case QAbstractSocket::HostNotFoundError:
        QMessageBox::information(this, "Error", "The host was not found");
        ui->pbConnect->setEnabled(true);
        break;
    case QAbstractSocket::ConnectionRefusedError:
        QMessageBox::information(this, "Error", "The connection was refused by
the peer.");
        ui->pbConnect->setEnabled(true);
        break;
    default:
        QMessageBox::information(this, "Error", "The following error occurred:
"+_sok->errorString());
        ui->pbConnect->setEnabled(true);
        break;
    }
}
```

```
void Dialog::onSokReadyRead()
{
    //обрабатываем данные от сервера
    QDataStream in(_sok);
    //если считываем новый блок первые 2 байта это его размер
    if (_blockSize == 0) {
        //если пришло меньше 2 байт ждем пока будет 2 байта
        if (_sok->bytesAvailable() < (int)sizeof(quint16))
            return;
        //считываем размер (2 байта)
        in >> _blockSize;
        qDebug() << "_blockSize now " << _blockSize;
    }
    //ждем пока блок прийдет полностью
    if (_sok->bytesAvailable() < _blockSize)
        return;
    else
        //можно принимать новый блок
        _blockSize = 0;
    //3 байт - команда серверу
    quint8 command;
    in >> command;
    qDebug() << "Received command " << command;

    switch (command)
    {
        //если удачно прошла авторизация, включаем кнопку отправки сообщения и
обновляем лог
        case MyClient::comAutchSuccess:
        {
            ui->pbSend->setEnabled(true);
            addToLog("Enter as "+_name,Qt::green);
        }
        break;

        //сервер отправит список пользователей, если авторизация пройдена, в
таком случае третий байт равен константе MyClient::comUsersOnline
        case MyClient::comUsersOnline:
        {
            addToLog("Received user list "+_name,Qt::green);
            ui->pbSend->setEnabled(true);
            ui->cbToAll->setEnabled(true);
            QString users;
            in >> users;
            if (users == "")
                return;

            //сервер передает имена через запятую, как строку (хотя можно писать
в поток и объект QStringList)
            QStringList l =  users.split(",");
```

```
        //обновляем гуи
        ui->lwUsers->addItems(l);
    }
    break;
    //общее сообщение от пользователя
    case MyClient::comMessageToAll:
    {
        //считываем и добавляем в лог
        QString user;
        in >> user;
        QString message;
        in >> message;
        addToLog("["+user+"]: "+message);
    }
    break;
    //ЛС от пользователя к пользователю
    case MyClient::comMessageToUsers:
    {
        //считываем и добавляем в лог
        QString user;
        in >> user;
        QString message;
        in >> message;
        addToLog("["+user+"](private): "+message, Qt::blue);
    }
    break;
    //общее сообщение от сервера
    case MyClient::comPublicServerMessage:
    {
        //считываем и добавляем в лог
        QString message;
        in >> message;
        addToLog("[Server]: "+message, Qt::red);
    }
    break;
    //ЛС от серва к пользователю
    case MyClient::comPrivateServerMessage:
    {
        QString message;
        in >> message;
        addToLog("[Server](private) : "+message, Qt::darkMagenta);
    }
    break;
    //Пользователь присоеденился
    case MyClient::comUserJoin:
    {
        QString name;
        in >> name;
        ui->lwUsers->addItem(name);
        addToLog(name+" joined", Qt::green);
    }
```

```
        break;
        //Пользователь ливнул
        case MyClient::comUserLeft:
        {
            QString name;
            in >> name;
            for (int i = 0; i < ui->lwUsers->count(); ++i)
                if (ui->lwUsers->item(i)->text() == name)
                {
                    ui->lwUsers->takeItem(i);
                    addToLog(name+" left", Qt::green);
                    break;
                }
        }
        break;
        //Если ник плохой
        case MyClient::comErrNameInvalid:
        {
            QMessageBox::information(this, "Error", "This name is invalid.");
            _sok->disconnectFromHost();
        }
        break;
        //Если ник занят
        case MyClient::comErrNameUsed:
        {
            QMessageBox::information(this, "Error", "This name is already
used.");
            _sok->disconnectFromHost();
        }
        break;
    }
}

//когда подключились
void Dialog::onSokConnected()
{
    ui->pbConnect->setEnabled(false);   //отрубаем кнопку подсоединения
    ui->leName->setEnabled(false);      //вырубаем возможность менять ник
    ui->leHost->setEnabled(false);      //вырубаем возможность менять IP
    ui->pbDisconnect->setEnabled(true); //врубаем кнопку отсоединения
    _blockSize = 0;
    addToLog("Connected to"+_sok-
>peerAddress().toString()+":"+QString::number(_sok->peerPort()),Qt::green);
//выводим сообщенеие о подключении

    //после подключения следует отправить запрос на авторизацию
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    //резервируем 2 байта для размера блока. Класс MyClient используется в
реализации сервера,
    //но тут используем статические члены этого класса - константы команд
```

```
    //третий байт - команда
    out << (quint16)0;
    out << (quint8)MyClient::comAutchReq;
    out << ui->leName->text();
    _name = ui->leName->text();

    //возваращаемся в начало
    out.device()->seek(0);
    //вписываем размер блока на зарезервированное место
    out << (quint16)(block.size() - sizeof(quint16));
    _sok->write(block);
}

//когда отключились
void Dialog::onSokDisconnected()
{
    ui->pbDisconnect->setEnabled(false);
    ui->pbSend->setEnabled(false);
    ui->cbToAll->setEnabled(false);
    ui->pbConnect->setEnabled(true);
    ui->leName->setEnabled(true);
    ui->leHost->setEnabled(true);
    ui->lwUsers->clear();
    addToLog("Disconnected from"+_sok->peerAddress().toString()+":3000",
Qt::green);
}

//по нажатию кнопки подключаемся к серверу
//connectToHost() возвращает тип void, потому что это асинхронный вызов и в
случае ошибки будет вызван слот onSokDisplayError
void Dialog::on_pbConnect_clicked()
{
    ui->pbConnect->setEnabled(false);
    _sok->connectToHost(ui->leHost->text(), PORT);
}

//по нажатию кнопки отключаемся от сервера
void Dialog::on_pbDisconnect_clicked()
{
    _sok->disconnectFromHost();
}

//меняем режим отправки сообщений
void Dialog::on_cbToAll_clicked()
{
    if (ui->cbToAll->isChecked())
        ui->pbSend->setText("Send");
    else
        ui->pbSend->setText("Send To Selected");
}
```

```
//отправляем сообщение
void Dialog::on_pbSend_clicked()
{
    //отправляем запрос на сообщение
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    //резервируем 2 байта для размера блока. Класс MyClient используется в
реализации сервера, но тут используем статические члены этого класса - константы
команд
    //третий байт - команда
    out << (quint16)0;
    if (ui->cbToAll->isChecked())
        out << (quint8)MyClient::comMessageToAll;
    else
    {
        out << (quint8)MyClient::comMessageToUsers;
        //записываем всех пользователей кому отправляем
        QString s;
        foreach (QListWidgetItem *i, ui->lwUsers->selectedItems())
            s += i->text()+",";
        s.remove(s.length()-1, 1);
        out << s;
    }
    //записываем сообщение
    out << ui->pteMessage->document()->toPlainText();
    //возваращаемся в начало
    out.device()->seek(0);
    //вписываем размер блока на зарезервированное место
    out << (quint16)(block.size() - sizeof(quint16));
    _sok->write(block);
    //очищаем поле сообщения
    ui->pteMessage->clear();
}

//выписываем в лог с нужным цветом
void Dialog::addToLog(QString text, QColor color)
{
    ui->lwLog->insertItem(0, QTime::currentTime().toString()+" "+text);
    ui->lwLog->item(0)->setForeground(QBrush(color));
}
```

Сервер

dialog.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>Dialog</class>
 <widget class="QWidget" name="Dialog">
  <property name="enabled">
   <bool>true</bool>
  </property>
  <property name="geometry">
   <rect>
    <x>0</x>
    <y>0</y>
    <width>600</width>
    <height>360</height>
   </rect>
  </property>
  <property name="minimumSize">
   <size>
    <width>600</width>
    <height>360</height>
   </size>
  </property>
  <property name="maximumSize">
   <size>
    <width>600</width>
    <height>360</height>
   </size>
  </property>
  <property name="windowTitle">
   <string>Omega server</string>
  </property>
  <property name="styleSheet">
   <string notr="true">background: qlineargradient(spread:pad, x1:1, y1:1, x2:0,
y2:0, stop:0 rgba(81, 0, 135, 255), stop:0.427447 rgba(41, 61, 132, 235), stop:1
rgba(155, 79, 165, 255));</string>
  </property>
  <widget class="QListWidget" name="lwUsers">
   <property name="geometry">
    <rect>
     <x>409</x>
     <y>60</y>
     <width>181</width>
     <height>141</height>
    </rect>
   </property>
   <property name="sizePolicy">
    <sizepolicy hsizetype="Minimum" vsizetype="Fixed">
     <horstretch>0</horstretch>
```

```
     <verstretch>0</verstretch>
    </sizepolicy>
   </property>
   <property name="styleSheet">
    <string notr="true">background:  rgb(255, 255, 255);
color: black;
border-radius: 5px;</string>
   </property>
   <property name="selectionMode">
    <enum>QAbstractItemView::MultiSelection</enum>
   </property>
   <property name="sortingEnabled">
    <bool>true</bool>
   </property>
  </widget>
  <widget class="QListWidget" name="lwLog">
   <property name="geometry">
    <rect>
     <x>10</x>
     <y>60</y>
     <width>381</width>
     <height>181</height>
    </rect>
   </property>
   <property name="sizePolicy">
    <sizepolicy hsizetype="Expanding" vsizetype="Expanding">
     <horstretch>0</horstretch>
     <verstretch>0</verstretch>
    </sizepolicy>
   </property>
   <property name="styleSheet">
    <string notr="true">background:  rgb(255, 255, 255);
color: black;
border-radius: 3px;</string>
   </property>
   <property name="selectionRectVisible">
    <bool>false</bool>
   </property>
  </widget>
  <widget class="QPlainTextEdit" name="pteMessage">
   <property name="enabled">
    <bool>true</bool>
   </property>
   <property name="geometry">
    <rect>
     <x>10</x>
     <y>270</y>
     <width>421</width>
     <height>71</height>
    </rect>
   </property>
```

```
   <property name="styleSheet">
    <string notr="true">background:  rgb(255, 255, 255);
color: black;
border-radius: 3px;</string>
   </property>
   <property name="plainText">
    <string/>
   </property>
  </widget>
  <widget class="QFrame" name="frame_2">
   <property name="geometry">
    <rect>
     <x>440</x>
     <y>270</y>
     <width>151</width>
     <height>74</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">background-color: rgba(255, 255, 255, 30);
border: 1px solid rgba(255, 255, 255, 40);
border-radius: 7px;</string>
   </property>
   <layout class="QVBoxLayout" name="verticalLayout_2">
    <item>
     <widget class="QCheckBox" name="cbToAll">
      <property name="enabled">
       <bool>false</bool>
      </property>
      <property name="toolTip">
       <string/>
      </property>
      <property name="toolTipDuration">
       <number>-1</number>
      </property>
      <property name="layoutDirection">
       <enum>Qt::LeftToRight</enum>
      </property>
      <property name="autoFillBackground">
       <bool>false</bool>
      </property>
      <property name="styleSheet">
       <string notr="true">QCheckBox{
    padding-left: 7px;
    color: rgb(255, 255, 255);
    background-color:rgba(255,255,255,30);
    border: 1px solid rgba(255,255,255,40);
    border-radius:7px;
    height: 22;
}</string>
      </property>
```

```
      <property name="inputMethodHints">
       <set>Qt::ImhLatinOnly</set>
      </property>
      <property name="text">
       <string>To ALL</string>
      </property>
      <property name="checked">
       <bool>true</bool>
      </property>
      <property name="tristate">
       <bool>false</bool>
      </property>
     </widget>
    </item>
    <item>
     <widget class="QPushButton" name="pbSend">
      <property name="enabled">
       <bool>false</bool>
      </property>
      <property name="styleSheet">
       <string notr="true">QPushButton{
    color: rgb(255, 255, 255);
    background-color:rgba(255,255,255,30);
    border: 1px solid rgba(255,255,255,40);
    border-radius:7px;
    height: 22;
}

QPushButton:hover{
    background-color:rgba(255,255,255,30);
}

QPushButton:pressed{
    background-color:rgba(255,255,255,70);
}</string>
      </property>
      <property name="text">
       <string>Send</string>
      </property>
     </widget>
    </item>
   </layout>
  </widget>
  <widget class="QPushButton" name="pbStartStop">
   <property name="geometry">
    <rect>
     <x>410</x>
     <y>240</y>
     <width>181</width>
     <height>23</height>
    </rect>
```

```
   </property>
   <property name="styleSheet">
    <string notr="true">QPushButton{
    color: rgb(255, 255, 255);
    background-color:rgba(255,255,255,30);
    border: 1px solid rgba(255,255,255,40);
    border-radius:7px;
    height: 22;
}

QPushButton:hover{
    background-color:rgba(255,255,255,30);
}

QPushButton:pressed{
    background-color:rgba(255,255,255,70);
}</string>
   </property>
   <property name="text">
    <string>Start server</string>
   </property>
   <property name="checkable">
    <bool>true</bool>
   </property>
   <property name="checked">
    <bool>false</bool>
   </property>
  </widget>
  <widget class="QLabel" name="chat_label">
   <property name="geometry">
    <rect>
     <x>10</x>
     <y>40</y>
     <width>91</width>
     <height>16</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
   </property>
   <property name="text">
    <string>Server logs</string>
   </property>
  </widget>
  <widget class="QLabel" name="usr_label">
   <property name="geometry">
    <rect>
```

```
      <x>410</x>
      <y>40</y>
      <width>141</width>
      <height>16</height>
     </rect>
    </property>
    <property name="styleSheet">
     <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
    </property>
    <property name="text">
     <string>Connected users</string>
    </property>
   </widget>
   <widget class="QLabel" name="msg_label">
    <property name="geometry">
     <rect>
      <x>10</x>
      <y>250</y>
      <width>81</width>
      <height>16</height>
     </rect>
    </property>
    <property name="styleSheet">
     <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
    </property>
    <property name="text">
     <string>Message</string>
    </property>
   </widget>
   <widget class="QLabel" name="label">
    <property name="geometry">
     <rect>
      <x>10</x>
      <y>10</y>
      <width>581</width>
      <height>31</height>
     </rect>
    </property>
    <property name="font">
     <font>
      <family>Monotype Corsiva</family>
      <pointsize>16</pointsize>
      <bold>true</bold>
```

```
    </font>
   </property>
   <property name="layoutDirection">
    <enum>Qt::LeftToRight</enum>
   </property>
   <property name="styleSheet">
    <string notr="true">color: white;
font-weight: bold;
font-size: 16pt;
background-color: none;
border: none;</string>
   </property>
   <property name="text">
    <string>SERVER WINDOW</string>
   </property>
   <property name="alignment">
    <set>Qt::AlignCenter</set>
   </property>
  </widget>
  <widget class="QLabel" name="usr_label_2">
   <property name="geometry">
    <rect>
     <x>410</x>
     <y>210</y>
     <width>74</width>
     <height>25</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">color: white;
font-weight: bold;
font-size: 12pt;
background-color: none;
border: none;</string>
   </property>
   <property name="text">
    <string> Server IP</string>
   </property>
  </widget>
  <widget class="QLineEdit" name="leHost">
   <property name="geometry">
    <rect>
     <x>488</x>
     <y>210</y>
     <width>101</width>
     <height>25</height>
    </rect>
   </property>
   <property name="styleSheet">
    <string notr="true">background:  rgb(255, 255, 255);
color: black</string>
```

```
   </property>
   <property name="text">
    <string>127.0.0.1</string>
   </property>
  </widget>
 </widget>
 <layoutdefault spacing="6" margin="11"/>
 <resources/>
 <connections/>
</ui>
```

dialog.h

```
#ifndef DIALOG_H
#define DIALOG_H

#include <QDialog>
#include <QDebug>
#include <QtGui>
#include <QtCore>
#include "myserver.h"
#include "myclient.h"

namespace Ui {
    class Dialog;
}

class Dialog : public QDialog
{
    friend class MyClient;
    Q_OBJECT

public:
    explicit Dialog(QWidget *parent = 0);
    ~Dialog();

private:
    Ui::Dialog *ui;
    MyServer *_serv;
    void addToLog(QString text, QColor color);

signals:
    void messageFromGui(QString message, const QStringList &users);

public slots:
    void onAddUserToGui(QString name);
    void onRemoveUserFromGui(QString name);
    void onMessageToGui(QString message, QString from, const QStringList &users);
    void onAddLogToGui(QString string, QColor color);
```

```
private slots:
    void on_pbSend_clicked();   // на Send кликнули
    void on_cbToAll_clicked();  // переключили режим отправки
    void on_pbStartStop_toggled(bool checked); // на включение/выключение
кликнули
};

#endif // DIALOG_H
```

dialog.cpp

```
#include "dialog.h"
#include "ui_dialog.h"
#include <QMessageBox>

#define PORT 3000

Dialog::Dialog(QWidget *parent) :QDialog(parent), ui(new Ui::Dialog)
{
    ui->setupUi(this);
    setWindowIcon(QIcon("favicon.PNG"));
    //создаем сервер. первый параметр стандартный - parent, второй - передадим
ссылку на объект виджета, для подключения сигналов от myclient к нему
    _serv = new MyServer(this, this);

    //подключаем сигналы от виджета к серверу
    connect(this, SIGNAL(messageFromGui(QString,QStringList)), _serv,
SLOT(onMessageFromGui(QString,QStringList)));
    connect(_serv, SIGNAL(addLogToGui(QString,QColor)), this,
SLOT(onAddLogToGui(QString,QColor)));

    //по умолчанию запускаем сервер на 127.0.0.1:PORT
    ui->pbStartStop->setChecked(true);
}

Dialog::~Dialog()
{
    delete ui;
}

void Dialog::onAddUserToGui(QString name)
{
    ui->lwUsers->addItem(name);
    ui->lwLog->insertItem(0, QTime::currentTime().toString()+" "+name+" joined");
    ui->lwLog->item(0)->setForeground(Qt::green);
}

void Dialog::onRemoveUserFromGui(QString name)
{
    for (int i = 0; i < ui->lwUsers->count(); ++i)
```

```
        if (ui->lwUsers->item(i)->text() == name)
        {
            ui->lwUsers->takeItem(i);
            ui->lwLog->insertItem(0, QTime::currentTime().toString()+" "+name+"
left");
            ui->lwLog->item(0)->setForeground(Qt::green);
            break;
        }
}

void Dialog::onMessageToGui(QString message, QString from, const QStringList
&users)
{
    if (users.isEmpty())
        ui->lwLog->insertItem(0, QTime::currentTime().toString()+" message from
"+from+": "+message+" to all");
    else
    {
        ui->lwLog->insertItem(0, QTime::currentTime().toString()+" message from
"+from+": "+message+" to "+users.join(","));
        ui->lwLog->item(0)->setForeground(Qt::blue);
    }
}

void Dialog::onAddLogToGui(QString string, QColor color)
{
    addToLog(string, color);
}

void Dialog::on_pbSend_clicked()
{
    QStringList l;
    if (!ui->cbToAll->isChecked())
        foreach (QListWidgetItem *i, ui->lwUsers->selectedItems())
            l << i->text();
    emit messageFromGui(ui->pteMessage->document()->toPlainText(), l);
    ui->pteMessage->clear();
    if (l.isEmpty())
        addToLog(" Sended public server message", Qt::black);
    else
        addToLog(" Sended private server message to "+l.join(","), Qt::black);
}

//меняем режим отправки сообщений
void Dialog::on_cbToAll_clicked()
{
    if (ui->cbToAll->isChecked())
        ui->pbSend->setText("Send");
    else
        ui->pbSend->setText("Send To Selected");
}
```

```
void Dialog::on_pbStartStop_toggled(bool checked)
{
    if (checked)
    {
        QHostAddress addr;
        if (!addr.setAddress(ui->leHost->text()))
        {
            addToLog(" invalid address "+ui->leHost->text(), Qt::red);
            return;
        }
        if (_serv->doStartServer(addr,PORT))
        {
            addToLog(" server started at "+ui->leHost->text()+":3000",
Qt::green);
            ui->pbSend->setEnabled(true);
            ui->cbToAll->setEnabled(true);
            ui->pbStartStop->setText("Stop server");
        }
        else
        {
            addToLog(" server not started at "+ui->leHost->text()+":3000",
Qt::red);
            ui->pbStartStop->setText("Reload");
            ui->pbSend->setEnabled(false);
            ui->cbToAll->setEnabled(false);
        }
    }
    else
    {
        addToLog(" server stopped at "+_serv->serverAddress().toString()+":3000",
Qt::green);
        _serv->close();
        ui->pbSend->setEnabled(false);
        ui->cbToAll->setEnabled(false);
        ui->pbStartStop->setText("Start server");

    }
}

void Dialog::addToLog(QString text, QColor color)
{
    ui->lwLog->insertItem(0, QTime::currentTime().toString()+text);
    ui->lwLog->item(0)->setForeground(color);
}
```

myclient.h

```
#ifndef MYCLIENT_H
#define MYCLIENT_H

#include <QObject>
#include <QDebug>
#include <QTcpSocket>
#include <QThreadPool>
#include <QtGui>
#include <QRegularExpression>
#include "dialog.h"
#include "myserver.h"
class MyServer;

class MyClient : public QObject
{
    friend class MyServer;
    Q_OBJECT

public:
    static const QString constNameUnknown;
    static const quint8 comAutchReq = 1;
    static const quint8 comUsersOnline = 2;
    static const quint8 comUserJoin = 3;
    static const quint8 comUserLeft = 4;
    static const quint8 comMessageToAll = 5;
    static const quint8 comMessageToUsers = 6;
    static const quint8 comPublicServerMessage = 7;
    static const quint8 comPrivateServerMessage = 8;
    static const quint8 comAutchSuccess = 9;
    static const quint8 comErrNameInvalid = 201;
    static const quint8 comErrNameUsed = 202;

    explicit MyClient(int desc, MyServer *serv, QObject *parent = 0);
    ~MyClient();
    void setName(QString name) {_name = name;}
    QString getName() const {return _name;}
    bool getAutched() const {return _isAutched;}
    void doSendCommand(quint8 comm) const;
    void doSendUsersOnline() const;

signals:
//сигналы для обновления гуи
    void addUserToGui(QString name);
    void removeUserFromGui(QString name);
//сигнал удаления пользователя из QList
    void removeUser(MyClient *client);
    void messageToGui(QString message, QString from, const QStringList &users);
//слоты для работы с сокетом
private slots:
```

```
    void onConnect();
    void onDisconnect();
    void onReadyRead();
    void onError(QAbstractSocket::SocketError socketError) const;

private:
    QTcpSocket *_sok;   //сокет
    MyServer *_serv;    //ссылка на сервер
    quint16 _blockSize; //текущей размер блока
    QString _name;      //имя
    bool _isAutched;    //флаг авторизации

};

#endif // MYCLIENT_H
```

myclient.cpp

```
#include "myclient.h"
#include <QWidget>
#include <QMessageBox>

const QString MyClient::constNameUnknown = QString(".Unknown");

MyClient::MyClient(int desc, MyServer *serv, QObject *parent) :QObject(parent)
{
    //храниим указатель на объект-сервер
    _serv = serv;
    //клиент не прошел авторизацию
    _isAutched = false;
    _name = constNameUnknown;
    //размер принимаемого блока 0
    _blockSize = 0;
    //создаем сокет
    _sok = new QTcpSocket(this);
    //устанавливаем дескриптор из incomingConnection()
    _sok->setSocketDescriptor(desc);
    //подключаем сигналы
    connect(_sok, SIGNAL(connected()), this, SLOT(onConnect()));
    connect(_sok, SIGNAL(disconnected()), this, SLOT(onDisconnect()));
    connect(_sok, SIGNAL(readyRead()), this, SLOT(onReadyRead()));
    connect(_sok, SIGNAL(error(QAbstractSocket::SocketError)), this,
SLOT(onError(QAbstractSocket::SocketError)));

    qDebug() << "Client connected" << desc;
}

MyClient::~MyClient()
{
```

```
}

void MyClient::onConnect()
{
    //не вызывается, т.к. сокет уже подключен к серваку (_sok-
>setSocketDescriptor(desc);)
    //надо чтоб работало
}

void MyClient::onDisconnect()
{
    qDebug() << "Client disconnected";
    //если авторизован
    if (_isAutched)
    {
        //убирием из интерфейса
        emit removeUserFromGui(_name);
        //сообщаем всем, что клиент вышел
        _serv->doSendToAllUserLeft(_name);
        //убираем из списка
        emit removeUser(this);
    }
    deleteLater();
}

void MyClient::onError(QAbstractSocket::SocketError socketError) const
{
    QWidget w;
    switch (socketError) {
    case QAbstractSocket::RemoteHostClosedError:
        break;
    case QAbstractSocket::HostNotFoundError:
        QMessageBox::information(&w, "Error", "The host was not found");
        break;
    case QAbstractSocket::ConnectionRefusedError:
        QMessageBox::information(&w, "Error", "The connection was refused by the
peer.");
        break;
    default:
        QMessageBox::information(&w, "Error", "The following error occurred:
"+_sok->errorString());
    }
}

void MyClient::onReadyRead()
{
    QDataStream in(_sok);
    //если считываем новый блок первые 2 байта это его размер
    if (_blockSize == 0) {
        //если пришло меньше 2 байт ждем пока будет 2 байта
        if (_sok->bytesAvailable() < (int)sizeof(quint16))
```

```
            return;
        //считываем размер (2 байта)
        in >> _blockSize;
        qDebug() << "_blockSize now " << _blockSize;
    }
    //ждем пока блок прийдет полностью
    if (_sok->bytesAvailable() < _blockSize)
        return;
    else
        //можно принимать новый блок
        _blockSize = 0;
    //3 байт - команда серверу
    quint8 command;
    in >> command;
    qDebug() << "Received command " << command;
    //для неавторизованный пользователей принимается только команда "запрос на
авторизацию"
    if (!_isAutched && command != comAutchReq)
        return;

    switch(command)
    {
        //запрос на авторизацию
        case comAutchReq:
        {
            //считываем имя
            QString name;
            in >> name;
            //проверяем его
            if (!_serv->isNameValid(name))
            {
                //отправляем ошибку
                doSendCommand(comErrNameInvalid);
                return;
            }
            //проверяем не используется ли имя
            if (_serv->isNameUsed(name))
            {
                //отправляем ошибку
                doSendCommand(comErrNameUsed);
                return;
            }
            //авторизация пройдена
            _name = name;
            _isAutched = true;
            //отправляем новому клиенту список активных пользователей
            doSendUsersOnline();
            //добавляем в интерфейс
            emit addUserToGui(name);
            //сообщаем всем про нового ползователя
            _serv->doSendToAllUserJoin(_name);
```

```
        }
        break;
        //от текущего пользователя пришло сообщение для всех
        case comMessageToAll:
        {
            QString message;
            in >> message;
            //отправляем его всем
            _serv->doSendToAllMessage(message, _name);
            //обновляем лог событий
            emit messageToGui(message, _name, QStringList());
        }
        break;
        //от текущего пользователя пришло сообщение для некоторых пользователей
        case comMessageToUsers:
        {
            QString users_in;
            in >> users_in;
            QString message;
            in >> message;
            //разбиваем строку на имена
            QStringList users = users_in.split(",");
            //отправляем нужным
            _serv->doSendMessageToUsers(message, users, _name);
            //обновляем интерфейс
            emit messageToGui(message, _name, users);
        }
        break;
    }

}

void MyClient::doSendCommand(quint8 comm) const
{
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    out << (quint16)0;
    out << comm;
    out.device()->seek(0);
    out << (quint16)(block.size() - sizeof(quint16));
    _sok->write(block);
    qDebug() << "Send to" << _name << "command:" << comm;
}

void MyClient::doSendUsersOnline() const
{
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    out << (quint16)0;
    out << comUsersOnline;
    QStringList l = _serv->getUsersOnline();
```

```
    QString s;
    for (int i = 0; i < l.length(); ++i)
        if (l.at(i) != _name)
            s += l.at(i)+(QString)",";
    s.remove(s.length()-1, 1);
    out << s;
    out.device()->seek(0);
    out << (quint16)(block.size() - sizeof(quint16));
    _sok->write(block);
    qDebug() << "Send user list to" << _name << ":" << s;
}
```

myserver.h

```
#ifndef MYSERVER_H
#define MYSERVER_H

#include <QTcpServer>
#include <QDebug>
#include "dialog.h"
#include "myclient.h"
class MyClient;

class MyServer : public QTcpServer
{
    Q_OBJECT

public:
    explicit MyServer(QWidget *widget = 0, QObject *parent = 0);
    bool doStartServer(QHostAddress addr, qint16 port);
    void doSendToAllUserJoin(QString name); //уведомить о новом пользователе
    void doSendToAllUserLeft(QString name);
    void doSendToAllMessage(QString message, QString fromUsername); //разослать
сообщение
    void doSendToAllServerMessage(QString message); //серверное сообщение
    void doSendServerMessageToUsers(QString message, const QStringList &users);
//приватное серверное сообщение
    void doSendMessageToUsers(QString message, const QStringList &users, QString
fromUsername);
    QStringList getUsersOnline() const; //узнать список пользователей
    bool isNameValid(QString name) const;   //проверить имя
    bool isNameUsed(QString name) const;    //проверить используется ли имя

signals:
    void addLogToGui(QString string, QColor color = Qt::black);
    void addUserToGui(QString);
    void removeUserFromGui(QString);

public slots:
    void onMessageFromGui(QString message, const QStringList &users);
```

```
    void onRemoveUser(MyClient *client);

protected:
    void incomingConnection(qintptr handle);

private:
    QList<MyClient *> _clients; //список пользователей
    QWidget *_widget;   //ссылка на виджет для подключения к нему сигналов от
myclient

};

#endif // MYSERVER_H
```

myserver.cpp

```
#include "myserver.h"

MyServer::MyServer(QWidget *widget, QObject *parent) :QTcpServer(parent)
{
    _widget = widget;
}

bool MyServer::doStartServer(QHostAddress addr, qint16 port)
{
    if (!listen(addr, port))
    {
        qDebug() << "Server not started at" << addr << ":" << port;
        return false;
    }
    qDebug() << "Server started at" << addr << ":" << port;
    return true;
}

//отправляем на клиенты инфу что кто-то присоеденился
//При рассылке сообщения всем нужно делать проверку авторизован ли текущий
пользователь, ибо в массиве _clients, возможно, находятся не авторизованные
void MyServer::doSendToAllUserJoin(QString name)
{
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    //резервируем 2 байта для размера блока. Класс MyClient используется в
реализации сервера, но тут используем статические члены этого класса - константы
команд
    //третий байт - команда
    out << (quint16)0 << MyClient::comUserJoin << name;
    //возваращаемся в начало
    out.device()->seek(0);
    //вписываем размер блока на зарезервированное место
```

```
    out << (quint16)(block.size() - sizeof(quint16));
    //отправка сообщения всем
    for (int i = 0; i < _clients.length(); ++i)
        if (_clients.at(i)->getName() != name && _clients.at(i)->getAutched())
            _clients.at(i)->_sok->write(block);
}

//отправляем на клиенты инфу что кто-то отсоединился
void MyServer::doSendToAllUserLeft(QString name)
{
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    out << (quint16)0 << MyClient::comUserLeft << name;
    out.device()->seek(0);
    out << (quint16)(block.size() - sizeof(quint16));
    for (int i = 0; i < _clients.length(); ++i)
        if (_clients.at(i)->getName() != name && _clients.at(i)->getAutched())
            _clients.at(i)->_sok->write(block);
}


//отправляем на клиенты сообщение от пользователя
void MyServer::doSendToAllMessage(QString message, QString fromUsername)
{
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    out << (quint16)0 << MyClient::comMessageToAll << fromUsername << message;
    out.device()->seek(0);
    out << (quint16)(block.size() - sizeof(quint16));
    for (int i = 0; i < _clients.length(); ++i)
        if (_clients.at(i)->getAutched())
            _clients.at(i)->_sok->write(block);
}

//отправляем на клиенты приватку от пользователя к выбраным пользователям
void MyServer::doSendMessageToUsers(QString message, const QStringList &users,
QString fromUsername)
{
    QByteArray block, blockToSender;
    QDataStream out(&block, QIODevice::WriteOnly);
    out << (quint16)0 << MyClient::comMessageToUsers << fromUsername << message;
    out.device()->seek(0);
    out << (quint16)(block.size() - sizeof(quint16));

    //отправка сообщения всем выбранным пользователям (тут отсутствует проверка
на авторизацию, ибо все пользователи в users гарантированно авторизованы)
    QDataStream outToSender(&blockToSender, QIODevice::WriteOnly);
    outToSender << (quint16)0 << MyClient::comMessageToUsers << users.join(",")
<< message;
    outToSender.device()->seek(0);
    outToSender << (quint16)(blockToSender.size() - sizeof(quint16));
```

```
    for (int j = 0; j < _clients.length(); ++j)
        if (users.contains(_clients.at(j)->getName()))
            _clients.at(j)->_sok->write(block);
        else if (_clients.at(j)->getName() == fromUsername)
            _clients.at(j)->_sok->write(blockToSender);
}

//отправляем на клиенты сообщение от сервера
void MyServer::doSendToAllServerMessage(QString message)
{
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    out << (quint16)0 << MyClient::comPublicServerMessage << message;
    out.device()->seek(0);
    out << (quint16)(block.size() - sizeof(quint16));
    for (int i = 0; i < _clients.length(); ++i)
        if (_clients.at(i)->getAutched())
            _clients.at(i)->_sok->write(block);
}

//отправляем на клиенты приватку от сервера к выбраным пользователям
void MyServer::doSendServerMessageToUsers(QString message, const QStringList
&users)
{
    QByteArray block;
    QDataStream out(&block, QIODevice::WriteOnly);
    out << (quint16)0 << MyClient::comPrivateServerMessage << message;
    out.device()->seek(0);
    out << (quint16)(block.size() - sizeof(quint16));

    //отправка сообщения всем выбранным пользователям (тут отсутствует проверка
на авторизацию, ибо все пользователи в users гарантированно авторизованы)
    for (int j = 0; j < _clients.length(); ++j)
        if (users.contains(_clients.at(j)->getName()))
            _clients.at(j)->_sok->write(block);
}


QStringList MyServer::getUsersOnline() const
{
    QStringList l;
    foreach (MyClient * c, _clients)
        if (c->getAutched())
            l << c->getName();
    return l;
}

bool MyServer::isNameValid(QString name) const
{
    if (name.length() > 20 || name.length() < 4)
        return false;
```

```
    QRegularExpression r("[A-Za-z0-9_]+");
    QRegularExpressionMatch match = r.match(name);
    return match.hasMatch();
}

bool MyServer::isNameUsed(QString name) const
{
    for (int i = 0; i < _clients.length(); ++i)
        if (_clients.at(i)->getName() == name)
            return true;
    return false;
}

void MyServer::incomingConnection(qintptr handle)
{
   //передаем дескриптор сокета, указатель на сервер (для вызова его методов), и
стандартный параметр - parent
    MyClient *client = new MyClient(handle, this, this);
    //подключаем сигналы напрямую к виджету, если его передали в конструктор
сервера
    if (_widget != 0)
    {
        connect(client, SIGNAL(addUserToGui(QString)), _widget,
SLOT(onAddUserToGui(QString)));
        connect(client, SIGNAL(removeUserFromGui(QString)), _widget,
SLOT(onRemoveUserFromGui(QString)));
        connect(client, SIGNAL(messageToGui(QString,QString,QStringList)),
_widget, SLOT(onMessageToGui(QString,QString,QStringList)));
    }
    connect(client, SIGNAL(removeUser(MyClient*)), this,
SLOT(onRemoveUser(MyClient*)));
    _clients.append(client);
}

void MyServer::onRemoveUser(MyClient *client)
{
    _clients.removeAt(_clients.indexOf(client));
}

void MyServer::onMessageFromGui(QString message, const QStringList &users)
{
    if (users.isEmpty())
        doSendToAllServerMessage(message);
    else
        doSendServerMessageToUsers(message, users);
}
```

main.cpp

```
#include "dialog.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    Dialog w;
    w.show();

    return a.exec();
}
```